Id: 2601240019

Name: Mogali Hema Durga Murari

Course: CTP_S2

Project Title: Multi-Factor Authentication Simulation System

Live Output URL: Not applicable – the system is a locally running Python application with an interactive frontend, not a hosted web service.

Evaluator Topic: Multi-Factor Authentication Flow, Challenge-Response Handling, Retry and Lockout Policies, and State-Machine Design

AI Tools Used

For this project, I used **Claude** (Anthropic) as an AI assistant to support understanding the authentication requirements, designing the object-oriented structure, developing the implementation, and preparing the tests and project documentation.

1. Claude as a Design and Coding Assistant

Claude was used to discuss how a multi-factor authentication flow can be modelled, to design the state machine and the class structure, to write and debug the Python code (including the asyncio challenge-response handling), to create pytest test cases, and to organise the frontend and storage components.

Why I Selected This AI Tool

- **Design support:** It helped me break the problem into clear states, transitions and policies before writing code.
- **Code-level assistance:** It supported development and debugging of the object-oriented and asynchronous parts of the project.
- **Testing support:** It helped me write pytest cases that check normal flows, failed attempts and lockout behaviour.
- **Documentation:** It helped me prepare explanations and project documents in a structured format.

Why Other AI Tools Were Not Selected as the Primary Tool

Tools such as ChatGPT, Google Gemini, GitHub Copilot and Cursor also provide coding and research support. I selected Claude as my primary tool because it suited my workflow of discussing the design first and then developing, testing and refining the project step by step in one continuous conversation.

Implementation Across the Project Requirements

1. Configurable Multi-Factor Authentication Flow

The system simulates an authentication process made up of multiple verification steps (factors). The sequence of factors is configurable, so the flow can be changed without rewriting the core logic. A user is authenticated only after all the required factors are successfully completed.

2. Object-Oriented State-Machine Design

The authentication process is modelled as a state machine with clearly defined states and allowed transitions (for example: started, factor pending, factor verified, authenticated, failed and locked). Each factor and the session are represented by separate classes, which keeps the design modular, readable and easy to extend.

3. Asynchronous Challenge-Response Handling

Challenges are issued and verified using Python's asyncio. Asynchronous handling allows the system to wait for user responses, apply time limits to a challenge, and manage more than one authentication session without blocking the program.

4. Retry and Lockout Policies

The system limits the number of failed attempts allowed. When the limit is reached, the session moves to a locked state according to the configured policy. This simulates how real authentication systems reduce the risk of repeated guessing.

5. Interactive Frontend

An interactive frontend allows the user to start an authentication session, respond to challenges, and view the current state and outcome of the process. The frontend uses the same underlying authentication logic as the rest of the project.

6. Storage

The project includes a storage component for keeping user and session information so that results and records are not lost when the application is closed. Keeping storage separate from the authentication logic makes it easier to change later.

7. Testing with pytest

Automated pytest test cases verify the main behaviours: successful authentication, wrong responses, retry limits, lockout, and invalid state transitions. Testing helps confirm that the state machine behaves as designed.

8. AI-Assisted Development and Quality Improvement

Claude supported the design discussion, code development, debugging, testing and documentation. Together with my own review and testing, this helped produce a modular, well-structured simulation of multi-factor authentication.

Conclusion

The Multi-Factor Authentication Simulation System demonstrates how a multi-step verification process can be designed using an object-oriented state machine, asynchronous challenge-response handling, and configurable retry and lockout policies. With an interactive frontend, storage and pytest-based testing, the project gives a practical and well-tested model of how multi-factor authentication works.

AI tools were used as development and learning assistants. The correctness of the final system depends on the actual code, its testing and my own review. This project is a simulation for learning and does not replace a production-grade authentication system.